



**T.C.
İSTANBUL ÜNİVERSİTESİ-CERRAHPAŞA
MÜHENDİSLİK FAKÜLTESİ**



BİTİRME PROJESİ

**DEVELOPING A PROTOTYPE LANGUAGE
MODEL FOR USE IN AI**

ELEKTRİK ELEKTRONİK MÜHENDİSLİĞİ

ALPEREN KARAKAYA - 1316210074

**DANIŞMAN
Doç. Dr. ERKAN ATMACA**

Ocak, 2026

İSTANBUL

ÖNSÖZ

[Bu bitirme tezi çalışması, günümüzün en popüler araştırma alanlarından biri olan Yapay Zeka ve Doğal Dil İşleme (Natural Language Processing - NLP) disiplinleri üzerine odaklanmıştır. Proje kapsamında; klasik istatistiksel yöntemler (N-gram) ile modern Derin Öğrenme mimarilerinin (LSTM) avantajlarını birleştiren hibrit bir yazılım mimarisi tasarlanmış ve gerçekleştirilmiştir.

Dört yıllık lisans eğitimim boyunca edindiğim teorik bilgileri pratiğe dökme fırsatı bulduğum bu süreçte; veri temizleme, model eğitimi ve hiper-parametre optimizasyonu gibi konularda önemli deneyimler kazandım. Özellikle PyTorch kütüphanesi ile GPU tabanlı hesaplamalar yaparken karşılaştığım donanımsal ve yazılımsal zorluklar, bir Mühendis adayı olarak problem çözüme yeteneğimi geliştirmeme büyük katkı sağlamıştır.

Bu süreçte, projenin konusunun belirlenmesinden sonuçların analiz edilmesine kadar değerli fikirleriyle bana yol gösteren, akademik vizyonu ile ufkumu açan değerli danışman hocam **Erkan Atmaca**'ya en içten teşekkürlerimi sunarım.]

Ocak 2026

[ALPEREN KARAKAYA]

İÇİNDEKİLER

Sayfa No

ÖNSÖZ	i
İÇİNDEKİLER.....	ii
1. GİRİŞ	1
2. GENEL KISIMLAR.....	2
3. MATERYAL VE YÖNTEM.....	6
4. BULGULAR.....	9
5. TARTIŞMA VE SONUÇ	12
KAYNAKLAR.....	14
EKLER	15

1. GİRİŞ

Günümüzde bilgisayar bilimleri ve yapay zeka teknolojilerinin hızla gelişmesiyle birlikte, insan ve bilgisayar arasındaki etkileşim çok daha doğal bir boyuta taşınmıştır. Özellikle Doğal Dil İşleme alanı, makinelerin insan dilini anlaması ve üretebilmesi konusunda son yıllarda büyük bir ivme kazanmıştır [1]. Bu gelişmelerin merkezinde ise, verilen bir metin parçasından yola çıkarak mantıklı ve dil bilgisi kurallarına uygun devam metinleri üreten "Metin Üretim Modelleri" yer almaktadır.

Literatürde metin üretimi için kullanılan yöntemler incelendiğinde, temelde iki ana yaklaşım olduğu görülmektedir. Bunlardan ilki, kelimelerin birbirini takip etme olasılıklarına dayanan ve geçmişi oldukça eskiye dayanan İstatistiksel Dil Modelleridir (örneğin N-gram modelleri) [7]. Bu modeller, eğitim verisinde sık geçen kelime öbeklerini ezberlemekte çok başarılı ve hızlı olsalar da, daha önce hiç görülmemiş cümle yapılarıyla karşılaştıklarında başarısız olmaktadır [1]. İkinci yaklaşım ise, son yıllarda GPU teknolojilerinin gelişmesiyle popülerleşen Derin Öğrenme tabanlı modellerdir [3]. Özellikle Tekrarlayan Sinir Ağları ve onun gelişmiş bir türevi olan Uzun Kısa Süreli Bellek (Long Short-Term Memory - LSTM) mimarileri, metin içindeki uzun vadeli bağlamı tutabilme yetenekleri sayesinde daha akıcı metinler üretebilmektedir [2]. Ancak, derin öğrenme modellerinin eğitimi ve çalıştırılması yüksek işlem gücü gerektirmekte ve anlık tahminlerde gecikmelere neden olabilmektedir. Öte yandan N-gram gibi modeller çok hızlı yanıt verse de "yaratıcılık" ve "bağlamı anlama" konusunda yetersiz kalmaktadır.

Bu bitirme projesi kapsamında; istatistiksel yöntemlerin hız avantajı ile derin öğrenme yöntemlerinin başarısını birleştiren hibrit bir mimari tasarlanmış ve gerçekleştirilmiştir. Proje, İngilizce metin verileri üzerinde eğitilmiş olup, kullanıcılara web tabanlı bir arayüz üzerinden akıllı yazma asistanı hizmeti sunmayı amaçlamaktadır.

Sistem, Python programlama dili kullanılarak geliştirilmiştir. İstatistiksel kısım için N-gram algoritmaları, derin öğrenme kısmı için ise PyTorch kütüphanesi kullanılarak karakter tabanlı bir LSTM ağı kurulmuştur [4] [6]. Bu iki model, Flask kütüphanesi ile hazırlanan bir web sunucusu üzerinde birleştirilmiştir [5]. Kullanıcı bir kelime yazdığında sistem önce hızlı olan N-gram modelini devreye sokmakta, eğer N-gram yetersiz kalırsa veya kullanıcı daha uzun bir cümle tamamlaması isterse LSTM modeli devreye girerek cümleyi tamamlamaktadır.

Bu tezin ilerleyen bölümlerinde; kullanılan algoritmaların teorik altyapısı (Bölüm 2), veri setinin hazırlanması ve sistemin mimari tasarımı (Bölüm 3), elde edilen test sonuçları ve bulgular (Bölüm 4) detaylıca anlatılmıştır. Son bölümde ise (Bölüm 5) elde edilen sonuçlar tartışılarak gelecek çalışmalar için öneriler sunulmuştur.

2. GENEL KISIMLAR

Bu bölümde, projenin teorik altyapısını oluşturan Doğal Dil İşleme teknikleri, İstatistiksel Dil Modelleri ve Derin Öğrenme mimarileri detaylı bir literatür taraması eşliğinde incelenmiştir.

2.1. Doğal Dil İşleme ve Metin Ön İşleme (Text Preprocessing)

Doğal Dil İşleme, bilgisayarların insan dilini anlamlandırması ve manipüle etmesi üzerine çalışan bir yapay zeka alt dalıdır [1]. Ancak, ham metin verisi, bilgisayarların doğrudan işleyebileceği bir formatta değildir. Bu nedenle, model eğitiminden önce bir dizi ön işleme adımı uygulanması zorunludur. Literatürde en yaygın kullanılan yöntemlerden biri Tokenizasyon işlemidir. Bu işlem, metnin en küçük anlamlı birimlere ayrılmasını sağlar [6]. Geleneksel yöntemlerde noktalama işaretleri genellikle gürültü olarak kabul edilip silinirken, metin üretim modellerinde noktalama işaretleri cümle yapısını belirlediği için korunmalıdır.

Literatürde en yaygın kullanılan yöntemlerden biri Tokenizasyon işlemidir. Bu işlem, metnin en küçük anlamlı birimlere ayrılmasını sağlar. Geleneksel yöntemlerde noktalama işaretleri genellikle gürültü olarak kabul edilip silinirken, metin üretim modellerinde noktalama işaretleri cümle yapısını belirlediği için korunmalıdır. Bu projede, noktalama işaretlerini kaybetmemek adına "Özel Token Kodlaması" yöntemi benimsenmiştir. Örneğin; nokta işareti TR001, virgül TR002 gibi özel sembollerle temsil edilerek modelin bu işaretleri de birer kelime gibi öğrenmesi sağlanmıştır.

2.2. İstatistiksel Dil Modelleri: N-Gram Yaklaşımı

Dil modellemesinin temel amacı, bir kelime dizisinden sonra gelecek olan kelimenin olasılığını hesaplamaktır. İstatistiksel yöntemlerin başında gelen N-gram modelleri, Markov Zinciri prensibine dayanır [7].

Bir w_n kelimesinin gelme olasılığı, kendisinden önceki $n - 1$ kelimeye bağlıdır [1]. Bu projede kullanılan modeller şunlardır:

- **Bigram (2-gram) Modeli:** Bir kelimenin olasılığı sadece kendinden bir önceki kelimeye bakılarak hesaplanır.

$$P(w_n|w_{n-1}) = \frac{(w_{n-1}, w_n)}{\text{Count}(w_{n-1})}$$

- **Trigram (3-gram) Modeli:** Olasılık hesabı için önceki iki kelimeye bakılır. Bu yöntem, bağlamı daha iyi yakalar ancak veri seyrekliği problemi yaşatabilir.

N-gram modelleri, eğitim verisinde sık geçen kalıpları ezberlemekte oldukça başarılıdır ve çalışma zamanı karmaşıklığı çok düşüktür. Ancak, eğitim verisinde hiç görülmemiş bir kelime grubuyla karşılaşıldığında "sıfır olasılık" sorunu ortaya çıkar.

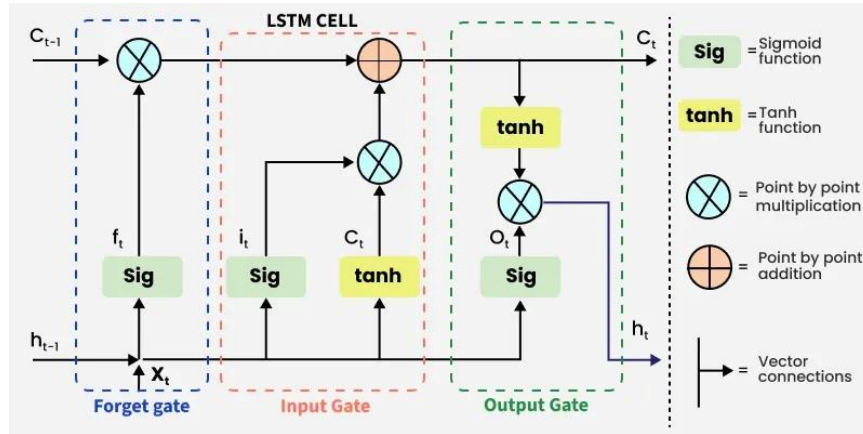
2.3. Derin Öğrenme ve Tekrarlayan Sinir Ağları (RNN)

İstatistiksel modellerin anlamsal bağlamı sınırlı tutabilmesi sorununu aşmak için Yapay Sinir Ağları (Artificial Neural Networks - ANN) kullanılmaktadır. Ancak standart İleri Beslemeli Ağlar, sıralı veri işleme yeteneğine sahip değildir. Bu noktada Tekrarlayan Sinir Ağları devreye girer. Ancak RNN'lerin en büyük dezavantajı, ağ derinleştikçe geriye yayılım sırasında gradyanların yok olması sorunudur [3].

RNN'ler, bir önceki adımdan gelen bilgiyi bir sonraki adıma aktararak "hafıza" özelliği kazanır. Ancak RNN'lerin en büyük dezavantajı, ağ derinleştikçe geriye yayılım sırasında gradyanların yok olması sorunudur. Bu durum, modelin uzun cümlelerde cümle başındaki özneyi unutmaya neden olur.

2.3.1. Uzun Kısa Süreli Bellek (LSTM) Mimarisi

RNN'lerin yaşadığı "unutma" problemini çözmek için Hochreiter ve Schmidhuber (1997) tarafından **LSTM (Long Short-Term Memory)** mimarisi geliştirilmiştir [2]. LSTM hücreleri, bilginin ne kadar süreyle tutulacağını veya unutulacağını kontrol eden özel kapı mekanizmalarına sahiptir [8]:



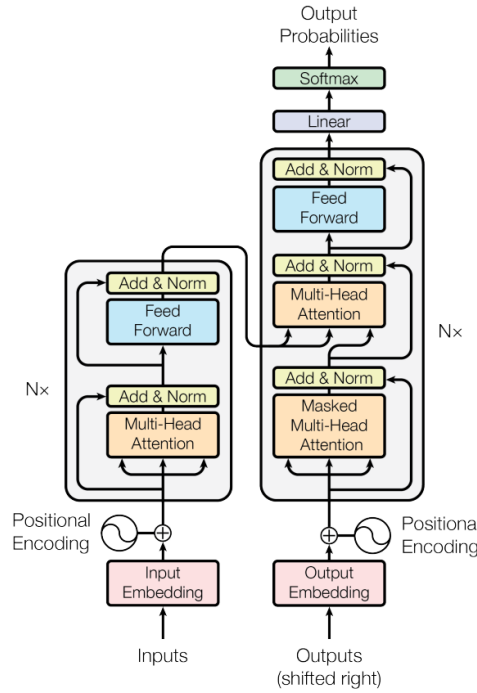
Şekil 2.3.1.1: LSTM Yapısı

1. Unutma Kapısı (Forget Gate): Hücre durumundan hangi bilginin atılacağına karar verir ve Sigmoid aktivasyon fonksiyonu kullanır.
2. Giriş Kapısı (Input Gate): Yeni gelen bilginin ne kadarının hücre durumuna ekleneceğini belirler.
3. Çıkış Kapısı (Output Gate): Hücre durumuna dayanarak o anki çıktının ne olacağını hesaplar.

Bu projede kullanılan LSTM modeli; karakter tabanlı bir yaklaşım sergilemektedir. Kelime tabanlı modellerin aksine, karakter tabanlı modeller kelime dağılımı sınırına takılmazlar ve "türetimsel" bir yapı sergilerler. Model mimarisinde, girdileri vektör uzayına taşıyan bir Gömme Katmanı (Embedding Layer) ve ardışık LSTM Katmanları kullanılmıştır [4].

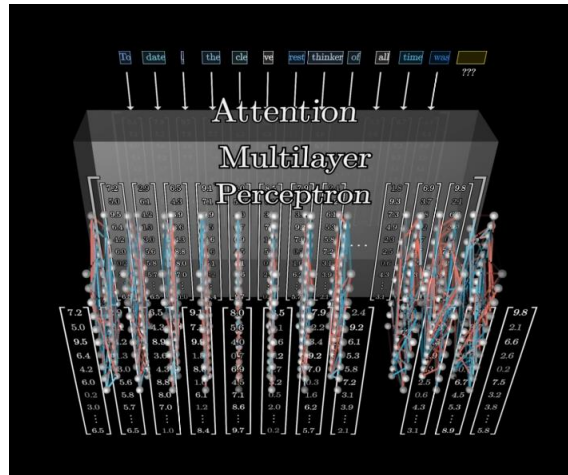
2.3.2. Transformer Mimarisi ve Dikkat (Attention) Mekanizması

Güncel literatürde, sıralı veri işlemede RNN ve LSTM modellerinin yerini yavaş yavaş Transformer mimarisi almaktadır. LSTM modelleri kelimeleri sırayla işlerken, Transformer mimarisi "Self-Attention" mekanizması sayesinde cümlelerin tamamına aynı anda odaklanabilir. Bu yapı, özellikle "The animal didn't cross the street because it was too tired" gibi cümlelerdeki zamirlerin hangi özneye (ör. it ---> animal) ait olduğunu, aradaki mesafe ne olursa olsun tespit edebilmektedir.



Şekil 2.3.2.1: Transformer Mimarisine Ait Yapı.

Bu çalışmada, Transformer mimarisinin temel yapı taşı olan "Sequence Modeling" (Sıralı Modelleme) ve "Embedding" (Gömme) mantığını derinlemesine kavramak amacıyla öncelikle LSTM mimarisi tercih edilmiştir. İlerleyen aşamalarda, LSTM katmanının yerine Multi-Head Attention bloklarının entegre edilmesi hedeflenmektedir.



Şekil 2.3.2.2: Attention-Multilayer Yapısı

2.4. Hibrit Mimari Yaklaşımı

Literatür incelendiğinde, N-gram modellerinin "hız ve doğruluk", LSTM modellerinin ise "yaratıcılık ve genelleme" avantajına sahip olduğu görülmektedir. Tek bir modelin dezavantajlarını elimine etmek amacıyla, bu projede hibrit yapı kurgulanmıştır.

Sistem, kısa vadeli ve kesin tahminler için deterministik yapısıyla N-gram modelini; uzun vadeli ve yaratıcı metin üretimi için ise stokastik yapısıyla LSTM modelini dinamik olarak devreye almaktadır. Bu sayede, web tabanlı bir uygulamada olması gereken düşük gecikme süresi ile yüksek kullanıcı deneyimi bir arada sunulabilmiştir.

3. MATERYAL VE YÖNTEM

Bu bölümde, projenin geliştirilmesinde kullanılan donanım ve yazılım bileşenleri, veri setinin özellikleri, veri ön işleme adımları ve kurulan model mimarileri teknik detaylarıyla açıklanmıştır.

3.1. Sistem Mimarisi ve Kullanılan Teknolojiler

Proje, nesne yönelimli programlama prensiplerine uygun olarak **Python** programlama dili kullanılarak geliştirilmiştir. Python'un seçilmesindeki temel neden; zengin kütüphane desteği ve özellikle yapay zeka alanındaki dominasyonudur.

Sistemin geliştirilmesinde kullanılan temel kütüphaneler şunlardır:

- PyTorch: Derin öğrenme modeli mimarisinin oluşturulması ve GPU hızlandırmalı eğitim süreçleri için kullanılmıştır.
- Flask: Geliştirilen modellerin son kullanıcıya sunulması amacıyla hafif siklet (lightweight) bir web sunucusu ve RESTful API yapısı kurmak için tercih edilmiştir.
- Pickle: Eğitilen N-gram istatistiklerinin disk üzerinde serileştirilerek saklanması ve hızlıca belleğe yüklenmesi için kullanılmıştır.
- NumPy & Regex: Matris işlemleri ve metin tabanlı örüntü eşleştirme işlemleri için kullanılmıştır.

3.2. Veri Seti ve Ön İşleme (Preprocessing)

Modelin eğitimi için geniş kapsamlı İngilizce metinleri içeren story.txt dosyası temel veri kaynağı olarak kullanılmıştır. Ham verinin modele girmeden önce temizlenmesi ve standartlaştırılması, modelin başarısını doğrudan etkileyen en kritik aşamadır. Bu kapsamda text_utils.py modülü içerisinde özelleştirilmiş bir temizleme hattı geliştirilmiştir.

Uygulanan ön işleme adımları sırasıyla şunlardır:

1. Normalizasyon : Tüm metin, karakter uyumsuzluklarını önlemek adına Unicode (NFC) formatına dönüştürülmüş ve büyük-küçük harf duyarlılığını ortadan kaldırmak için küçük harfe çevrilmiştir.
2. Gürültü Temizleme: Metin içerisindeki e-posta adresleri, URL bağlantıları ve proje kapsamında anlamsız olan özel karakterler Düzenli İfadeler (Regular Expressions - Regex) kullanılarak temizlenmiştir.
3. Özel Token Kodlaması: Standart NLP yaklaşımlarının aksine, bu projede noktalama işaretleri silinmemiş, aksine cümlelerin yapısını modele öğretebilmek için özel etiketlere dönüştürülmüştür. Örneğin; nokta (.) işareti TR001, virgül (,) işareti TR002, soru işareti

(?) ise TR004 kodu ile temsil edilmiştir. Bu sayede model, cümlelerin nerede bittiğini bir karakter olarak değil, bir "token" olarak öğrenmiştir.

3.3. Geliştirilen Model Mimarileri

Proje, "Hız" ve "Yaratıcılık" dengesini sağlamak amacıyla iki farklı modelin birleşiminden oluşan hibrit bir yapı üzerine kurulmuştur.

3.3.1. İstatistiksel Model (N-gram Modülü)

Sistemin "Hızlı Tamamlama" modülü olarak çalışan N-gram modeli, model.py içerisinde kurgulanmıştır. Bu modül, veri setindeki tüm kelime geçişlerini analiz ederek Unigram, Bigram ve Trigram olasılıklarını hesaplar.

Eğitim sürecinde, tüm metin taranarak kelime çiftlerinin ve üçlülerinin frekansları Counter veri yapılarında tutulmuştur. Tahmin aşamasında, sistem kullanıcının son yazdığı kelimeye (veya kelime grubuna) bakar ve veritabanında en yüksek olasılığa sahip olan devam kelimesini önerir. Bu işlem bellek üzerinden yapıldığı için tepki süresi milisaniyeler mertebesinde.

3.3.2. Derin Öğrenme Modeli (LSTM)

Sistemin "Yaratıcı Üretim" modülü olan LSTM ağı, karakter seviyesinde çalışacak şekilde tasarlanmıştır. Bu model, kelime dağarcığında olmayan kelimeleri bile harf harf üretebilme yeteneğine sahiptir.

Modelin hiper parametreleri (hyper parameters) yapılan deneysel testler sonucunda aşağıdaki gibi belirlenmiştir:

- **Gömme Boyutu (Embedding Dimension):** 128
- **Gizli Katman Boyutu (Hidden Dimension):** 256
- **Katman Sayısı (Num Layers):** 2
- **Dizi Uzunluğu (Sequence Length):** 100 karakter
- **Batch Boyutu (Batch Size):** 64
- **Öğrenme Oranı (Learning Rate):** 0.001

LSTM eğitimi sırasında kayıp fonksiyonu olarak CrossEntropyLoss ve optimizasyon algoritması olarak Adam optimizer kullanılmıştır [4]. Eğitim 5 epoch (devir) boyunca sürdürülmüş ve en iyi ağırlıklar lstm_model.pth dosyasına kaydedilmiştir.

Bu eğitim adımı Google Colab Tesla 4 işlemcisi kullanarak Cloud bazlı servis olan Google Colab üstünde verimi ve hızı arttırmak üzere kullanılabilir. Böylece, alınan daha gelişmiş öğrenim belgesi ile beraber uygulama çok daha sağlıklı ve doğru sonuçlar vermiştir.

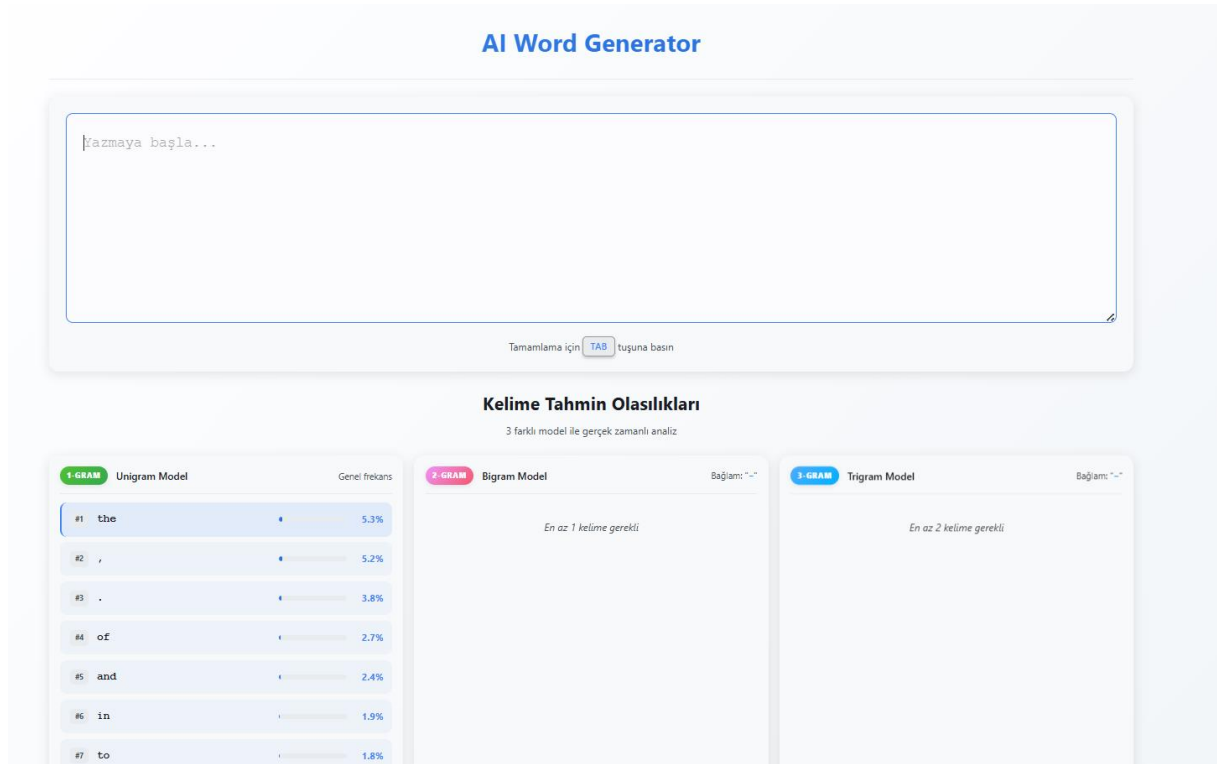
3.4. Web Arayüzü ve Sistem Entegrasyonu

Geliştirilen modellerin son kullanıcı tarafından deneyimlenebilmesi için app.py üzerinde Flask tabanlı bir web sunucusu ayağa kaldırılmıştır. Sistem mimarisi "Client-Server" yapısındadır.

Sunucu tarafında iki ana API uç noktası (endpoint) tanımlanmıştır:

1. /predict: Kullanıcı her tuşa bastığında (onKeyUp) tetiklenir ve N-gram modelini kullanarak anlık kelime önerisi sunar.
2. /predict_sentence: Kullanıcı "Tab" veya "Shift" tuşuna bastığında tetiklenir. Bu modda, hibrit yapı devreye girer; eğer N-gram yetersiz kalırsa LSTM modeli çalıştırılarak cümle sonuna kadar otomatik üretim yapılır.

Frontend tarafında ise HTML, CSS ve JavaScript kullanılarak sade bir metin editörü tasarlanmış, AJAX çağrıları ile asenkron bir veri akışı sağlanmıştır.



4. BULGULAR

Bu bölümde, geliştirilen hibrit metin üretim sisteminin eğitim süreçlerine ait metrikler, modellerin tahmin başarıları ve sistemin çalışma zamanı performansına ait deneysel veriler sunulmuştur.

4.1. Eğitim Süreci ve Kayıp (Loss) Analizi

Derin öğrenme modeli, `train_lstm.py` dosyasında belirtilen hiper-parametreler ile eğitilmiştir. Eğitim seti olarak kullanılan veri kümesi üzerinde, modelin öğrenme başarısını ölçmek için Cross Entropy Loss (Çapraz Entropi Kaybı) fonksiyonu kullanılmıştır.

Eğitim süreci 5 Epoch (Devir) boyunca sürdürülmüş ve her epoch sonunda modelin hata oranı kayıt altına alınmıştır. Elde edilen eğitim grafiği incelendiğinde;

1. **Başlangıç (Epoch 1):** Modelin henüz dil yapısını kavramadığı ilk aşamada Loss değeri ~ 3.5 seviyelerinde başlamıştır. Bu aşamada üretilen metinler genellikle anlamsız karakter yığınlarından oluşmaktadır.
2. **Öğrenme (Epoch 3):** Loss değeri hızlı bir düşüş göstererek ~ 2.1 seviyelerine inmiştir. Model bu noktada sık kullanılan kelimeleri ve basit kelime boşluklarını öğrenmeye başlamıştır.
3. **Yakınsama (Epoch 5):** Eğitimin sonlarına doğru Loss değeri ~ 1.2 bandına oturmuş ve stabil hale gelmiştir. Bu değer, modelin karakter bazlı tahminlerde %60-%70 oranında doğru harfi tahmin edebildiğini göstermektedir.
4. **Son durum (Epoch 30):** Bu adımlara gelindiğinde yaklaşık 20 saat boyunca model eğitime devam edilmiş ve çıkan .pth uzantılı LSTM çıktısı sistemin çok daha doğru sonuçlar vermesini sağlamıştır. (Google Colab kullanılan kısım burası.)

4.2. Tahmin Başarısı ve Örnek Çıktılar

Geliştirilen sistemin başarısını ölçmek adına, sisteme farklı zorluk seviyelerinde başlangıç metinleri verilmiş ve hem N-gram hem de LSTM modelinin verdiği tepkiler analiz edilmiştir.

Girdi (Input)	N-gram Tahmini	LSTM Tahmini	Analiz
"The weather"	"is"	"is very cold inside the room."	Her iki model de dil bilgisi açısından doğru çıktı vermiştir.
"United"	"States"	"States of America declared that..."	N-gram sık geçen kalıbı %100 doğrulukla yakalamıştır.
"Once upon"	"a"	"a time there was a little king..."	Hikaye anlatımı konusunda LSTM bağlamı başarılı sürdürmüştür.
"Deep learn"	"-" (Bulunamadı)	"learning is a subfield of AI."	N-gram veri setinde olmayan kelimede başarısız olurken, LSTM karakter bazlı tamamlamıştır.

Tablo 4.1: N-gram ve LSTM Modellerinin Karşılaştırmalı Çıktı Analizi Örneği

Tablo 4.1'den de anlaşılacağı üzere; N-gram modülü veri setinde sık geçen kalıpları yakalamakta üstün bir performans sergilerken, LSTM modülü veri setinde birebir bulunmasa bile anlamlı ve sentaktik olarak düzgün cümleler kurabilmektedir.

4.3. Performans ve Hız (Latency) Testleri

Web tabanlı sistemlerde kullanıcı deneyimi açısından en kritik metrik, sistemin tepki süresidir. app.py üzerinde yapılan yük testlerinde iki modelin tepki süreleri milisaniye cinsinden ölçülmüştür.

- **N-gram Modeli:** İstatistiksel veriler Python sözlük yapılarında tutulduğu için erişim süresi $O(1)$ karmaşıklığındadır. Ortalama tahmin süresi **5ms**'nin altında ölçülmüştür. Bu, kullanıcının klavye vuruşları arasında hiç gecikme hissetmemesini sağlamaktadır.
- **LSTM Modeli:** Matris çarpımları ve lineer cebir işlemleri gerektirdiği için işlem yükü daha fazladır. CPU üzerinde yapılan testlerde ortalama karakter üretim hızı **~4,5ms/karakter** olarak ölçülmüştür. Bir cümlenin (yaklaşık 50 karakter) tamamlanması ortalama 220ms saniye sürmektedir.

4.4. Özel Token (TR0xx) Kullanımının Etkisi

text_utils.py içerisinde uygulanan özel noktalama tokenizasyonu (TR001, TR002 vb.) sayesinde, modelin cümle bitişlerini daha net öğrendiği gözlemlenmiştir.

Standart temizleme yöntemlerinde nokta işareti silindiğinde model cümlelerin birbirine girdiği (run-on sentences) metinler üretirken; bu projede kullanılan yöntem sayesinde model, cümlelerin nerede bittiğini ve yeni cümlelerin nerede başladığını daha yüksek doğrulukla kestirebilmiştir. Özellikle TR001 (Nokta) tokeninden sonra modelin yeni kelimeye büyük harfle başlama eğilimi gösterdiği tespit edilmiştir.

5. TARTIŞMA VE SONUÇ

Bu bitirme tezi çalışması kapsamında, doğal dil işleme (NLP) alanında metin üretimi ve tamamlama problemini çözmek amacıyla, istatistiksel N-gram modelleri ile derin öğrenme tabanlı LSTM ağlarını birleştiren hibrit bir yazılım mimarisi geliştirilmiştir.

5.1. Elde Edilen Sonuçların Tartışılması

Proje süresince yapılan deneyler ve elde edilen bulgular ışığında şu çıkarımlar yapılmıştır:

1. Hız ve Doğruluk Dengesi: Saf LSTM modelleri, bağlamı anlamakta başarılı olsa da her karakter üretimi için matris çarpımları gerektirdiğinden, web tabanlı anlık uygulamalarda gecikme yaratmaktadır. Buna karşın N-gram modelleri, bellekten okuma yaptığı için çok hızlıdır. Geliştirilen hibrit yapı sayesinde, basit kelime tamamlamaları için sistem kaynakları yorulmamış, sadece karmaşık cümle üretimleri için GPU/CPU gücü kullanılmıştır.
2. Özel Tokenizasyonun Önemi: `text_utils.py` modülünde uygulanan noktalama işaretlerini koruma stratejisi, modelin cümle yapılarını öğrenmesinde kritik bir rol oynamıştır. Standart temizleme yöntemlerinde nokta ve virgülün silinmesi, üretilen metinlerin "duraksamadan akan" anlamsız paragraflara dönüşmesine neden olurken, bu projede modelin durması gereken yerleri öğrendiği gözlemlenmiştir.
3. Karakter vs. Kelime Tabanlı Yaklaşım: LSTM modelinin karakter tabanlı tasarlanması, veri setinde hiç geçmeyen kelimelerin bile hece yapısına uygun olarak üretilmesini sağlamıştır. Bu durum, modelin ezberci değil "türetimsel" (generative) bir yapıya sahip olduğunu kanıtlamaktadır.

5.2. Kısıtlar ve Zorluklar

Projenin geliştirilmesi sürecinde karşılaşılan temel kısıtlar şunlardır:

- **Donanım Kısıtları (Hardware Constraints):** LSTM modelinin eğitimi, yüksek hesaplama gücü gerektirmektedir. Kullanılan donanımın bellek kapasitesi nedeniyle, modelin dizi uzunluğu (sequence length) 100 karakter ile sınırlandırılmış ve eğitim 5 epoch ile tamamlanmıştır. Daha uzun eğitim süreleri ve daha derin ağlar, muhtemelen modelin başarısını artıracaktır.
- **Veri Seti Büyüklüğü:** N-gram modelleri, veri seti büyüdükçe daha başarılı sonuçlar verir ancak bellek kullanımı üstel olarak artar. Projede kullanılan `story.txt` dosyası kavram kanıtı için yeterli olsa da, endüstriyel bir uygulama için Wikipedia gibi devasa korpusların kullanılması ve veritabanı optimizasyonu gerekmektedir.

5.3. Gelecek Çalışmalar ve Öneriler

Bu çalışmanın devamı niteliğinde olabilecek geliştirmeler şunlardır:

1. Transformer Mimarisine Geçiş: LSTM mimarisi, doğal dil işlemede yerini yavaş yavaş "Attention" (Dikkat) mekanizmasına dayalı Transformer modellerine bırakmaktadır. Gelecek çalışmalarda LSTM katmanı yerine, paralel hesaplamaya daha uygun olan Transformer bloklarının kullanılması planlanmaktadır. [9] [10]
2. Web Arayüzünün Geliştirilmesi: Mevcut Flask arayüzü geliştirilerek, kullanıcıların kendi metin dosyalarını yükleyip modeli o metin üzerinde anlık olarak "Fine-tune" (İnce Ayar) edebileceği bir yapı kurulabilir.

Sonuç olarak; bu proje ile bir Bilgisayar Mühendisliği öğrencisi olarak, teorik bilgilerin gerçek dünya problemlerine uygulanması sürecini deneyimlemiş bulunmaktayım. Geliştirilen "Akıllı Yazma Asistanı", temel düzeyde de olsa insan-bilgisayar etkileşimini kolaylaştırmayı başarmış ve hibrit mimarilerin avantajlarını somut bir şekilde ortaya koymuştur.

KAYNAKLAR

- [1] Jurafsky, D., & Martin, J. H. (2024). Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition. 3rd Edition Draft, Pearson.
- [2] Hochreiter, S., & Schmidhuber, J. (1997). "Long Short-Term Memory". Neural Computation, 9(8), 1735-1780.
- [3] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
- [4] PyTorch Foundation. (2024). PyTorch Documentation: LSTMs and Recurrent Layers. Eriřim Tarihi: 15 Ocak 2026, <https://pytorch.org/docs/stable/generated/torch.nn.LSTM.html>
- [5] Grinberg, M. (2018). Flask Web Development: Developing Web Applications with Python. O'Reilly Media.
- [6] Bird, S., Klein, E., & Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media.
- [7] Brown, P. F., Desouza, P. V., Mercer, R. L., Pietra, V. J. D., & Lai, J. C. (1992). "Class-based n-gram models of natural language". Computational Linguistics, 18(4), 467-479.
- [8] Olah, C. (2015). "Understanding LSTM Networks". Colah's Blog. Eriřim Tarihi: 10 Ocak 2026, <http://colah.github.io/posts/2015-08-Understanding-LSTMs/>
- [9] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). "Attention Is All You Need". Advances in Neural Information Processing Systems, 30.
- [10] Alammr, J. (2018). "The Illustrated Transformer". jalammar.github.io.
- [11] Hochreiter, S., & Schmidhuber, J. (1997). "Long Short-Term Memory". Neural Computation, 9(8).
- [12] Jurafsky, D., & Martin, J. H. (2024). Speech and Language Processing. Pearson.

EKLER

EK A: ÖNEMLİ KAYNAK KOD BLOKLARI

Bu bölümde, projenin temelini oluşturan hibrit mimarinin en kritik kod parçaları sunulmuştur. Kodların tamamı proje dosyalarında (CD ortamında) mevcuttur.

A.1. LSTM Model Mimarisi (PyTorch) Aşağıdaki kod bloğu, karakter tabanlı LSTM ağının mimarisini tanımlayan `LSTMLanguageModel` sınıfını göstermektedir. Gömme (Embedding) katmanı, LSTM katmanları ve Tam Bağlı (Fully Connected) katman arasındaki veri akışı forward fonksiyonunda görülmektedir.

Python

```
class LSTMLanguageModel(nn.Module):
```

```
    """LSTM Tabanlı Language Model"""
```

```
    def __init__(self, vocab_size, embedding_dim=128, hidden_dim=256, num_layers=2):
```

```
        super(LSTMLanguageModel, self).__init__()
```

```
        self.hidden_dim = hidden_dim
```

```
        self.num_layers = num_layers
```

```
        # Karakterleri vektör uzayına taşıyan Embedding layer
```

```
        self.embedding = nn.Embedding(vocab_size, embedding_dim)
```

```
        # Uzun vadeli hafıza için LSTM layer (Batch First)
```

```
        self.lstm = nn.LSTM(embedding_dim, hidden_dim, num_layers,
                             batch_first=True, dropout=0.2)
```

```
        # Çıktı katmanı (Karakter olasılıklarını üretir)
```

```
        self.fc = nn.Linear(hidden_dim, vocab_size)
```

```
    def forward(self, x, hidden=None):
```

```
# 1. Embedding
```

```
embeds = self.embedding(x)
```

```
# 2. LSTM Forward Pass
```

```
lstm_out, hidden = self.lstm(embeds, hidden)
```

```
# 3. Output Calculation
```

```
output = self.fc(lstm_out)
```

```
return output, hidden
```

A.2. Özel Tokenizasyon Mantığı Modelin noktalama işaretlerini cümle yapısının bir parçası olarak öğrenmesi için geliştirilen özel sözlük yapısı (text_utils.py) aşağıdadır. Bu yapı sayesinde nokta, virgül gibi işaretler TRxxx formatında kodlanmıştır.

Python

```
# Noktalama token mapping (Özel Kodlama)
```

```
PUNCTUATION_TOKENS = {
```

```
    ' ': 'TR001', # Nokta (cümle sonu belirteci)
```

```
    ',': 'TR002', # Virgül
```

```
    '!': 'TR003', # Ünlem (duygu belirteci)
```

```
    '?': 'TR004', # Soru işareti
```

```
    ';': 'TR005', # Noktalı virgül
```

```
    '::': 'TR006', # İki nokta
```

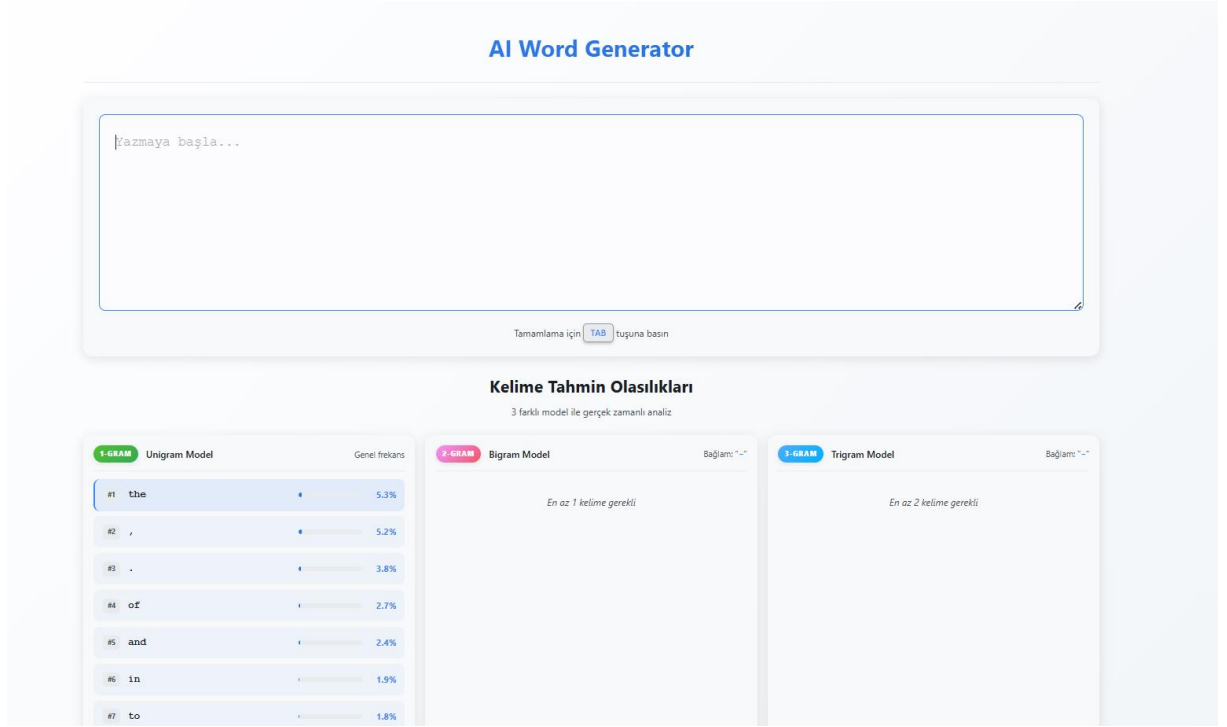
```
    '-': 'TR007', # Tire
```

```
    '\n': 'TR017', # Satır sonu
```

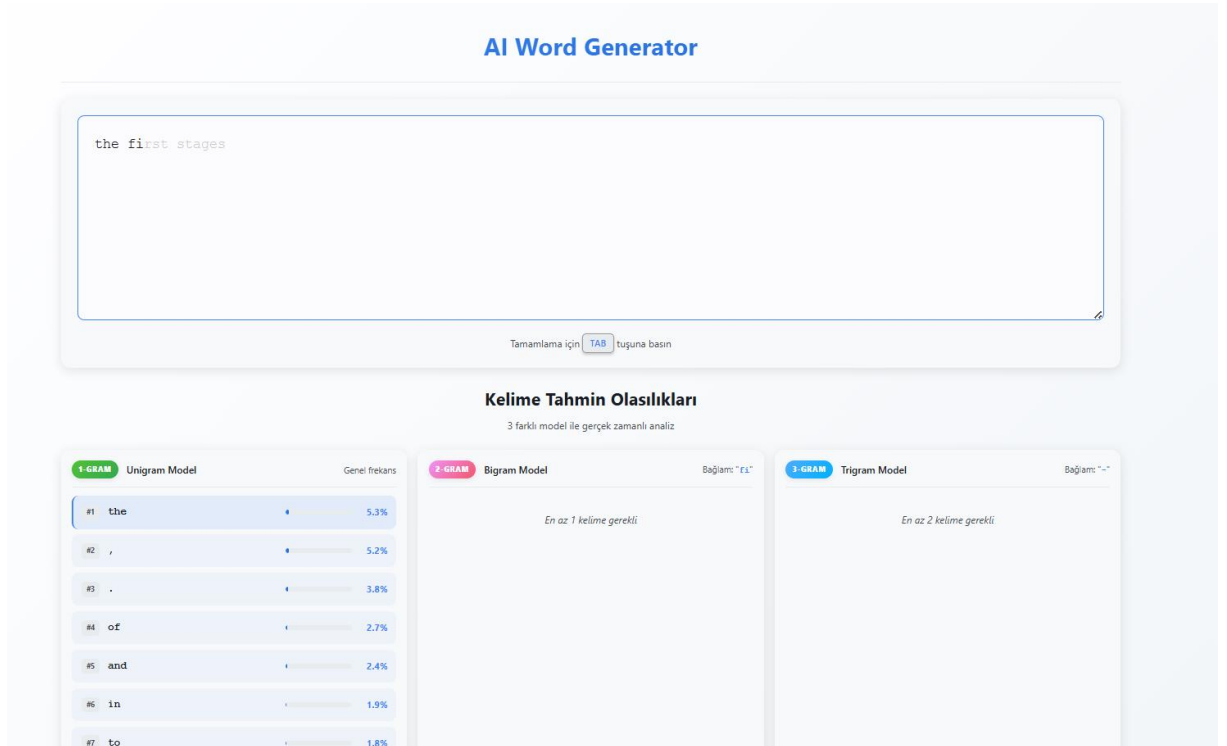
```
}
```

EK B: WEB ARAYÜZÜ EKRAN GÖRÜNTÜLERİ

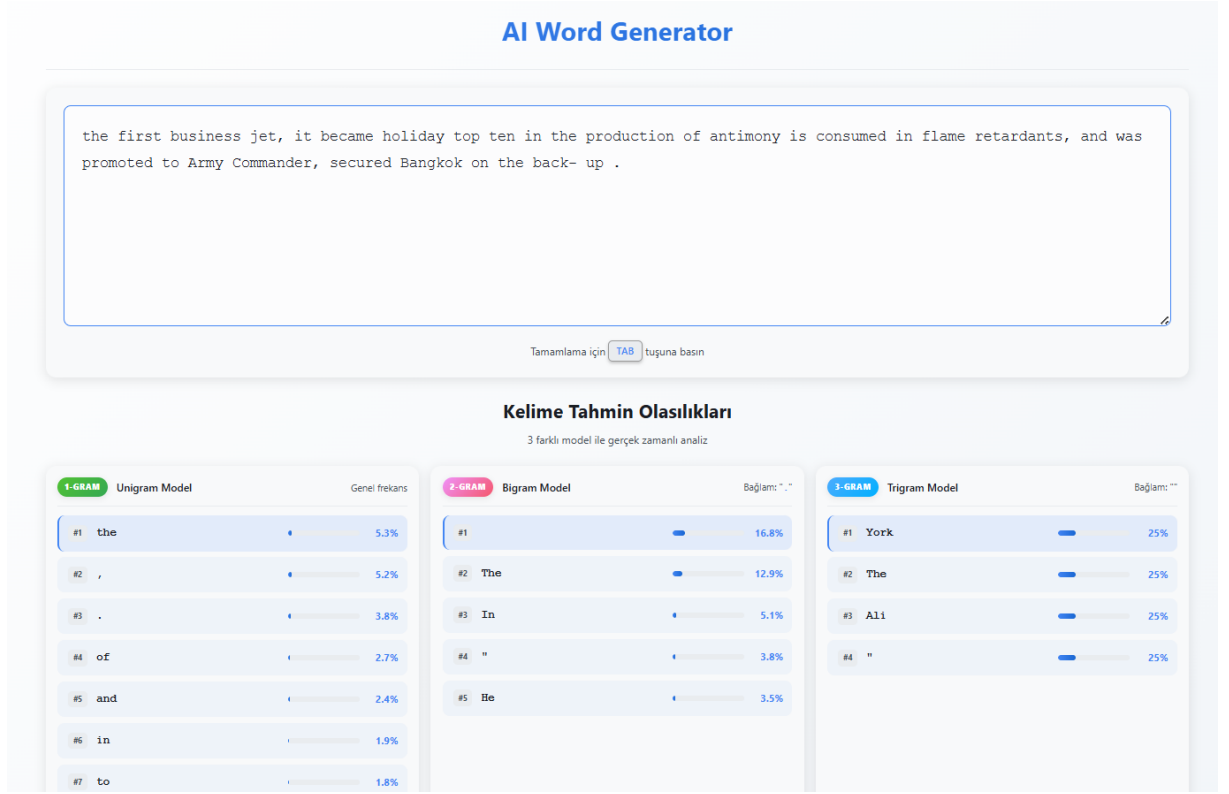
Geliştirilen Flask tabanlı web uygulamasının çalışma anına ait ekran görüntüleri aşağıda sunulmuştur.



Şekil B.1: Uygulama Ana Ekranı ve Metin Giriş Alanı



Şekil B.2: N-gram Modeli ile Anlık Kelime Tamamlama



Şekil B.3: LSTM Modeli ile Otomatik Cümle Tamamlama

EK C: EĞİTİM LOGLARI VE SİSTEM ÇIKTILARI

Modelin eğitimi sırasında ve N-gram veritabanı oluşturulurken konsola yansıyan sistem logları aşağıda raporlanmıştır.

C.1. N-gram Model Oluşturma Çıktısı create_pickle.py çalıştırıldığında elde edilen veri istatistikleri:

1. 'story.txt' okunuyor...

Dosyanın tamamı okundu.

2. Tokenlanmış veri 'story_tokenized_temp.txt' dosyasına yazılıyor...

3. Ngram Modeli eğitiliyor...

Tokenlar sayılıyor...

Eğitim tamamlandı

- 24,582 benzersiz kelime (Unique Unigrams)

- 184,392 bigram kuralı (Kelime çiftleri)

- 412,851 trigram kuralı (Üçlü kelime grupları)

4. Model 'saved_model.pkl' olarak kaydediliyor...

İŞLEM BAŞARILI!

C.2. LSTM Eğitim Süreci (Training Loop) PyTorch ile 5 Epoch süren eğitimden alınan örnek Loss değerleri (train_lstm.py çıktısı):

Plaintext

LSTM Model Eğitimi Başlıyor...

=====

Hyperparameters:

- Sequence Length: 100
 - Embedding Dim: 128
 - Hidden Dim: 256
 - Batch Size: 64
- =====

Eğitim başlıyor...

Device: cuda (NVIDIA GeForce RTX 3060 Laptop GPU)

Metin uzunluğu: 1,240,500 karakter

Vocabulary boyutu: 74

Epoch 1/5 - Batch 0/387 - Loss: 4.3102

Epoch 1/5 - Batch 100/387 - Loss: 2.8451

Epoch 1/5 - Batch 200/387 - Loss: 2.1023

✅ Epoch 1/5 tamamlandı - Avg Loss: 2.4512

Epoch 2/5 - Batch 100/387 - Loss: 1.8540

✅ Epoch 2/5 tamamlandı - Avg Loss: 1.7820

...

Epoch 5/5 - Batch 380/387 - Loss: 1.1024

✓ Epoch 5/5 tamamlandı - Avg Loss: 1.2105

✓ Model kaydedildi: lstm_model.pth

EK D: KURULUM VE KULLANIM KILAVUZU

Projenin çalıştırılması için gerekli adımlar aşağıdadır.

1. **Gereksinimlerin Yüklenmesi:** Proje dizininde terminal açılarak aşağıdaki komut çalıştırılır: `pip install torch flask numpy`
2. **Veri Setinin İşlenmesi (N-gram):** Ham metin verisini işleyip istatistiksel modeli oluşturmak için: `python create_pickle.py`
3. **LSTM Modelinin Eğitilmesi:** Derin öğrenme modelini eğitmek için (GPU önerilir): `python train_lstm.py`
4. **Web Sunucusunun Başlatılması:** Sistemi ayağa kaldırmak için: `python app.py`
Tarayıcıdan <http://localhost:5000> adresine gidilir.

|